ReMaP: Macro Placement by Recursively Prototyping and Periphery-Guided Relocating

Yunqi Shi[1,2,3*], Xi Lin[1,2*], Siyuan Xu[3], Shixiong Kai[3], Ke Xue[1,2], Mingxuan Yuan[3], Chao Qian[1,2], Zhi-Hua Zhou[1,2]
[1]National Key Laboratory for Novel Software Technology, Nanjing University, China
[2]School of Artificial Intelligence, Nanjing University, China
[3]Huawei Noah's Ark Lab, China

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132944

***Abstract*—We introduce the ReMaP framework, which generates expert-quality macro placements through recursively prototyping and periphery-guided relocating. A key innovation is ABPlace, an angle-based analytical method that arranges macros along an ellipse to facilitate a rough distribution near the periphery, while optimizing dataflow, minimizing overlap, and ensuring convergence. Based on the results of ABPlace, an efficient heuristic is proposed to position macros along the chip's periphery, mirroring practices often employed by experts. Our framework outperforms three leading macro placers in both WNS and TNS across eight test cases, achieving improvements up to 34.15% in WNS and 65.39% in TNS, as tested on the popular OpenROAD-flow-scripts infrastructure. Additionally, our parameter auto-tuning method further improves timing by 8.75%.**

I. Introduction

The macro placement problem has been a significant focus in both academia and industry for decades. Ineffective macro placement can obstruct essential core areas of a chip, leading to overcrowded and elongated paths that delay timing and complicate the positioning of standard cells. This issue has become increasingly important as chips grow in size and complexity, requiring the integration of more macros, which makes achieving efficient placement more challenging.

Due to its critical role, macro placement has traditionally required experts who use specialized knowledge for each specific case. This includes optimizing dataflow to enhance timing, placing macros along chip periphery to reduce routing congestion, tiling macros to improve power management, and positioning macros away from pin areas to facilitate pin access [9], [10]. Experts also use feedback from back-end evaluations and front-end engineers to refine the results iteratively, which can take weeks or months to complete [19].

To achieve expert-quality placement results and reduce turnaround time (TAT), automated placement algorithms emulate expert behaviors and incorporate heuristics into their optimization processes. Our work focuses on positioning macros along the chip periphery, and incorporating specific strategies to enhance placement regularity. It's important to distinguish between periphery placement and regularity, as they serve different purposes. Some approaches address them separately [9], [14], while others do not [22], [26]. Periphery placement helps preserve central regions for standard cell placement, reducing routing congestion and detours, which enhances overall timing performance. Improving regularity involves tiling macros of the same size and minimizing notch and dead areas [9], [14], which not only benefits power planning but also helps placement and routing for large-scale, high-utilization cases.

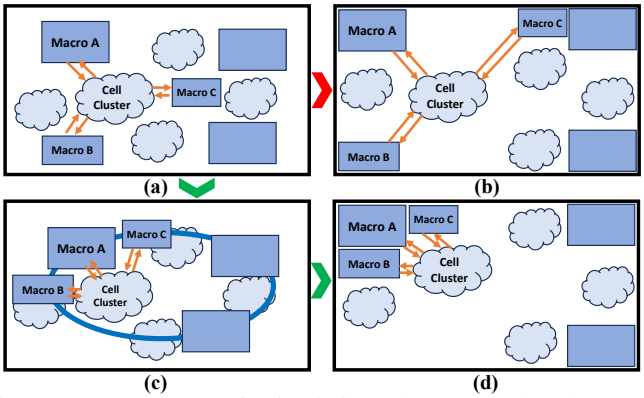


Fig. 1: (a) Prototype of mixed-size placement. (b) Placement result following the direct application of the periphery-pushing heuristic. (c) Optimization of macros on an ellipse, called ABPlace. (d) Periphery-guided relocating of macros based on their placement along the ellipse.

The general motivation and strategy of our flow is shown in Fig. 1. We begin with a mixed-size prototyping as shown in Fig. 1(a). An intuitive yet less effective method, shown in Fig. 1(b), involves moving each macro towards the nearest corner or periphery of the chip layout. No matter where we place the cell cluster, the total communication cost remains high due to the considerable distances between macros A, B, and C. Alternatively, to relocate macros to the periphery, we fix all cell clusters and optimize macro positions along an ellipse, considering dataflow cost and macro overlap, as shown in Fig. 1(c). The macros are then strategically moved to optimal locations considering dataflow and periphery cost, as illustrated in Fig. 1(d). This sequence of steps (a), (c), and (d) is repeated recursively. The number of macros placed at each iteration is tuned to balance efficiency and accuracy.

Our contributions are summarized as follows:

- We develop a macro placement framework, **ReMaP**, that delivers expert-quality results through a recursive process of prototyping and periphery-guided relocating. Our algorithm naturally handles pre-placed macro scenarios by iteratively placing and fixing macros in stages. We have open-sourced the code at https://github.com/lamda-bbo/DAC25-ReMaP and provided detailed evaluation scripts and metadata to facilitate replication.
- We introduce ABPlace, an innovative angle-based ana-

* Equal contribution.

lytical method that arranges and optimizes macros along an ellipse. This approach considers the dataflow strength between macros and macro-cell clusters, and helps distribute macros uniformly near the chip periphery. With a single angle variable to optimize for each macro, ABPlace achieves easy convergence across all test cases.
- We design an efficient heuristic to relocate macros to the chip periphery, guided by the results from ABPlace. We propose two criteria to determine the placement order: one emphasizes dataflow, while the other enhances layout regularity. By dividing the feasible placement region into grids, we then optimize dataflow through greedy selection.
- Our extensive comparisons with three leading macro placement tools—RTL-MP [9], Hier-RTLMP [10], and DREAMPlace 4.1.0 with macro placement enhancements [2], [18]—demonstrate that the proposed ReMaP achieves superior post-route WNS and TNS across all eight chip cases from the OpenROAD-flow-scripts [1]. We also involve Bayesian optimization for parameter autotuning and performance improvement.

II. Related Works

Generally, macro placement techniques can be classified into three categories: zeroth-order methods, analytical methods, and learning-based methods.

Zeroth-order methods are extensively utilized in the field of design automation, as most of the power, performance and area (PPA) metrics are black-box and non-differentiable. There are two fundamental components for a zeroth-order method for macro placement: solution representation and evaluation. Initial research efforts mainly focus on proposing effective data structures to represent a placement solution, including corner block list [7], sequence pair [20], B*-tree [25], slicing tree [27], and so on. The evaluation criteria mainly include wirelength and total area. Then zeroth-order optimization algorithms like simulated annealing or evolutionary algorithms are applied to optimize the solution. Recently, dataflow has emerged as a prominent optimization target [24], offering insights into RTL-level information. It aims to reveal the underlying connection relationships between macros [28]. Specifically, RTL-MP [9] and Hier-RTLMP [10] incorporate multilevel clustering through dataflow extraction to enhance placement regularity and performance. These methods employ a weighted sum of seven distinct targets as the optimization objective, effectively integrating engineering know-how. Another representative work by Lin et al. [15] uses corner stitching [21] to represent the solution, effectively addressing the challenges associated with pre-placed blocks and enhancing overall efficiency.

Analytical methods [2], [3], [17], [18] try to distribute macros and cells simultaneously by optimizing the wirelength and placement density. Post-processing steps such as macro adjustment or legalization are then applied to further enhance the placement. These methods have the merit of efficiency and the global view of wirelength. Recently, DG-RePlAce [11] and DAPA [16] try to involve dataflow as a constraint into the global placement, trying to capture the critical datapaths and optimize them accordingly. Another recent work IncreMacro [22] identifies macros placed away from the periphery and incorporates smoothed cost as a nonlinear optimization metric.

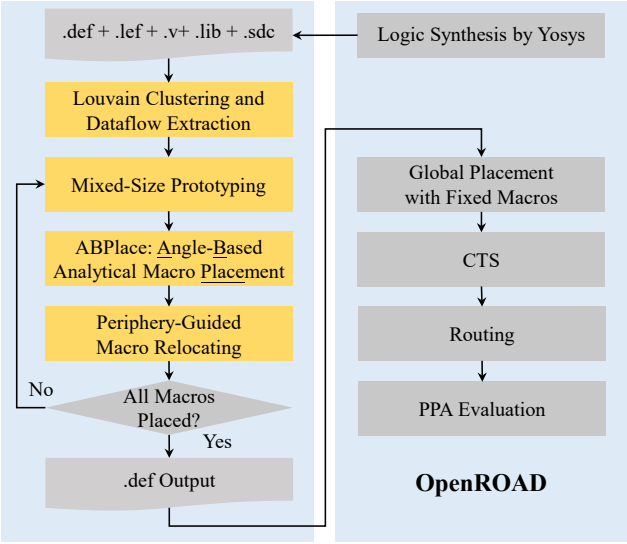


Fig. 2: Overflow of our proposed ReMaP method. Components in yellow are key steps of ReMaP.

Learning-based methods have recently gained attention, starting with Google's Graph Placement [19]. These methods employ reinforcement learning for macro placement, yet face challenges with reward shaping. Providing rewards only after all macros are placed [4], [19] leads to sparse feedback, while using incremental wirelength as a dense reward [6], [12], [13], [23] often misaligns with PPA metrics.

Despite advancements, the macro placement challenge remains largely unresolved. Traditional zeroth-order methods may fail to capture the locations of standard cells after global placement. Recent approaches like RTL-MP [9] and Hier-RTLMP [10] attempt to address this by involving dataflow and using a sequence pair to simultaneously represent macros and cell clusters. However, the positions of cell clusters determined by simulated annealing may not align with those from an actual global placement. Analytical methods are also widely adopted, but they face convergence issues for mixed-size designs. Moreover, further refining and legalizing macros typically involve heuristics [2], [22], which can lead to inevitable performance degradation when all standard cells are treated as fixed [22].

III. Proposed ReMap Framework

In this section, we introduce ReMaP, a novel macro placement framework that generates expert-quality macro placement layouts. As shown in Fig. 2, our process begins with a synthesized netlist, from which we perform clustering and extract dataflow connections among macros and cell clusters. We employ DREAMPlace 4.1.0 [2], [18] to establish a mixed-size placement prototype, setting initial positions for each macro and cell cluster. It is followed by ABPlace, an angle-based analytical method that optimizes macro positions on a predefined ellipse (or a circle for square chip canvas). We then relocate macros to optimize the dataflow and periphery cost on a discretized canvas. In each iteration, we process a subset

of macros, fix their positions, and cycle the remaining macros back into the loop for next iteration. The final PPA evaluation is performed using the full OpenROAD-flow-scripts [1].

A. Clustering and Dataflow Extraction

To reduce complexity and uncover connections among macros and cells, we begin our placement process with clustering. We use star decomposition to break down nets into individual edges, forming an undirected graph. The modularity-based Louvain algorithm [5] then identifies communities within this graph, while also considering pre-placed I/O pins. For precise macro placement, each macro is extracted and treated as its own cluster. After forming all clusters, we traverse the initial graph to calculate the **direct** dataflow strength between each cluster pair, acquiring the dataflow matrix A. The process above is only performed once during the entire placement process.

Although several hierarchical clustering and dataflow extraction methods have been proposed [8]–[10], we opt for the straightforward Louvain algorithm and focus on direct connections for dataflow analysis, which have already yielded superior results. Our ReMaP framework can be integrated with any advanced clustering method for enhanced performance.

B. Mixed-Size Prototyping

Incorporating cell locations into macro placement is crucial, as these combinational components occupy significant space and connect macros. Traditional macro placers often model cell clusters as soft blocks and optimize them alongside macros. However, determining the area and shape of these clusters is challenging, and the final cell positions may differ after analytical placement. Direct mixed-size global placement can also face divergence issues, leading to suboptimal results.

To accurately estimate cell cluster positions and avoid divergence, we adopt a self-adaptive analytical mixed-size placement approach. Initially, a high target density facilitates convergence when many macros are movable. As more macros become fixed, the target density decreases to align with the low-density design's final cell placement[1]. Using this mixed-size prototype, we can determine the locations of macros and cell clusters by averaging the positions of all cells within each cluster.

C. ABPlace: Angle-Based Analytical Macro Placement

After acquiring the positions of macros and cell clusters, we fix the cell clusters and map each macro to the boundary of an ellipse. Let the width and height of the chip core be W and H. The center of the ellipse aligns with the chip's center, both at the origin. The ellipse's semi-major a and semi-minor b axes are scaled by β and shrink with each iteration by γ. Thus, the ellipse's equation at the k-th iteration is:

$$\begin{cases} a = \beta\gamma^k W/2,\ b = \beta\gamma^k H/2 \\ x^2/a^2 + y^2/b^2 = 1 \end{cases}$$

where x and y describe the coordinates of points on the ellipse. When $W = H$, the ellipse becomes a circle. To map macros onto the ellipse, we draw a ray from the ellipse center to the macro center. The intersection point with the ellipse is the macro's new position. Given the original coordinates (x_i^0, y_i^0) of macro i, the mapped position (x_i, y_i) is calculated by:

$$\theta_i = \arctan \frac{y_i^0}{x_i^0},\ x_i = a\cos\theta_i,\ y_i = b\sin\theta_i,$$

where $\theta_i \in [0, 2\pi)$ represents the angle of macro i.

After mapping, we fix cell clusters and optimize macro positions using angle-based analytical optimization. With macros restricted to the ellipse, we only optimize the angle vector $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} \left(\sum_{i\in M}\sum_{j\in N} \text{dist}(i,j)A_{i,j} + \lambda \sum_{i,j\in M, i\neq j} \text{overlap}(i,j) \right), \tag{1}$$

where A is the dataflow matrix indicating connection strength, M is the set of all unplaced macros, and N includes all macros and cell clusters. We use Euclidean distance for dist function. The overlap penalty between macros i and j is defined as:

$$\begin{aligned} \text{overlap}(i,j) = &\max\{(w_i + w_j)/2 - |x_i - x_j|, 0\} \\ &* \max\{(h_i + h_j)/2 - |y_i - y_j|, 0\}, \end{aligned}$$

where w_i and h_i are the width and height of macro i, respectively. The $\max$ function has discontinuities at zero, which can complicate gradient calculations. We use `torch.clamp` to handle the boundary conditions. Additionally, we approximate the absolute value using a smoothing function $\sqrt{x^2 + \epsilon^2} - \epsilon$ to maintain differentiability throughout the optimization process.

Considering our ultimate goal of placing macros regularly along the chip's periphery while minimizing data flow loss, ABPlace offers two key advantages by incorporating an ellipse: it positions macros close to their final periphery locations in a uniform manner while considering dataflow cost, and it allows for easy convergence without extensive parameter tuning.

D. Periphery-Guided Macro Relocating

After optimizing macro positions on the ellipse, we relocate certain macros to the chip's periphery, aiming to minimize dataflow loss. The problem is formulated as follows: given a chip layout with several placed macros, we relocate a set of macros previously positioned on the ellipse boundary, one by one. At each placement step, we solve two key problems: selecting which macro to place and determining its location.

We propose two criteria for selecting macros to place:

- **Descending order of macro's overlapping area**: Larger macros in crowded regions on the ellipse are prioritized for placement. This approach enhances regularity in two ways. First, placing larger macros near the periphery and smaller ones towards the core improves the utilization of outer space. Second, macros with similar shapes and sizes tend to be sorted and placed together.
- **Ascending order of dataflow strength**: Macros with weaker dataflow connections are placed first, allowing macros with stronger connections to be positioned closer to the core. This strategy reduces the negative impact on overall dataflow from periphery placement. Additionally,

[1]This dynamic approach is designed for OpenROAD-flow-scripts benchmarks [1] with low utilization and target density, as shown in Table I. We will extend ReMaP to larger, high-density designs in the future.

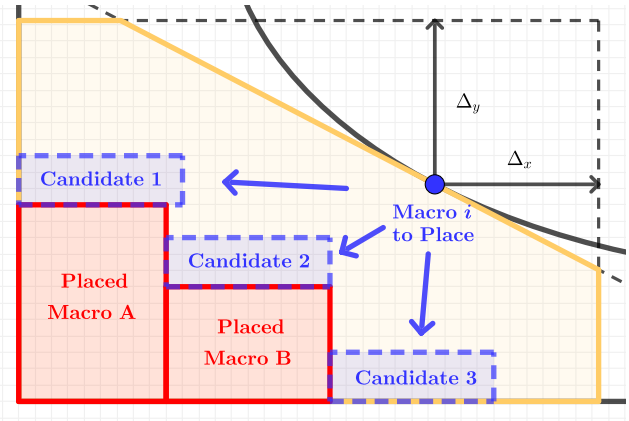


Fig. 3: Illustration of periphery-guided macro relocating: With macros A and B already placed in previous iterations, macro i is optimized on the ellipse by ABPlace in the current iteration. It will be placed in one of three candidate positions.

macros of the same size and group often share similar connection strengths, so they tend to be placed together, further enhancing placement regularity.

We choose the second criterion as our default setting for potentially better timing performance. The first criterion is also tested in ablation studies IV-B, producing layouts with higher regularity and occasionally better timing in specific cases. Overall, the second criterion generally performs better.

Next, we determine macro positions along the chip periphery inside a discretized feasible region. As illustrated in Fig. 3, we draw a tangent to the ellipse at macro i. Then we draw lines parallel to the x and y axes at distances Δ_x and Δ_y from the macro center. This forms a pentagon, shaded in yellow in Fig. 3, representing the placable region for macro i. We discretize this region into grids, and traverse feasible grids (x, y) that avoid overlaps, minimizing two cost functions c_{peri} and c_{df}:

$$c_{\text{peri}} = \min\{W - x, x\} + \min\{H - y, y\},$$
$$c_{\text{df}} = \sum\nolimits_{j \in P} \text{dist}(i, j) A_{i,j},$$

where c_{peri} and c_{df} represent the periphery cost and dataflow cost, respectively, and P includes placed macros and all cell clusters. We first consider c_{peri} using a Pareto dominance approach. Grid (x, y) dominates (x', y') if and only if:

$$\begin{cases} \min\{W - x, x\} \leq \min\{W - x', x'\} \\ \min\{H - y, y\} \leq \min\{H - y', y'\} \end{cases}$$

We select grids that are not dominated by others as candidates. Such selection prefers locations that are close to the chip corner. As shown in Fig. 3, only three candidate positions remain. We then compute c_{df} for each candidate grid, and choose the grid with the optimal (minimal) c_{df} value.

E. Summary and Implementation Details

The proposed ReMaP is presented in Algorithm 1. Lines 1–3 prepare the initial data for placement optimization. We extract only direct connections as the dataflow matrix, as discussed in Sec. III-A. While we attempted to incorporate indirect connections, like one-hop and two-hop links [10], [28], the improvement was negligible and it required further parameter tuning for these weaker connections. With accurate standard cell positions integrated into our optimization, direct connections suffice in our practice.

Algorithm 1 ReMaP Framework

Input: .def, .lef, .v, .lib, .sdc, decay factor γ, scale factor β, number k of macros to place per iteration, number n of macros.
Output: .def with all macros placed.
Process:

1: Cluster all macros and cells using the Louvain algorithm.
2: Extract dataflow matrix A.
3: Separate macros from clusters, treating each as an individual cluster.
4: $iter \leftarrow 0$
5: **while** there remain unplaced macros **do**
6: Perform a preliminary mixed-size placement and compute the centroid of each cell cluster.
7: $a \leftarrow \beta\gamma^{iter}W/2$, $b \leftarrow \beta\gamma^{iter}H/2$
8: Map each unplaced macro onto the ellipse defined by a and b.
9: Execute ABPlace for angle-based analytical placement.
10: Rank macros by specified criteria and select the top k.
11: **for** each macro in the top k **do**
12: Select the optimal grid considering dataflow and periphery cost.
13: Place and fix the macro.
14: **end for**
15: $iter \leftarrow iter + 1$
16: **end while**

Lines 5–16 iteratively optimize macro placement. In each iteration, k macros are placed and fixed. The parameter k balances efficiency and accuracy; setting k to 1 maximizes accuracy but increases runtime, especially for circuit with numerous macros. We typically set k to $\lceil n/10 \rceil$ to ensure that the algorithm completes within 10 iterations. The ellipse size is updated in each iteration. Initially, it is large to reach the periphery of the layout. As more macros are placed, the ellipse size reduces to fit the available space. We set the scale factor β to 0.9 and keep it fixed. The decay factor γ, however, is adjusted based on specific cases. During ABPlace in line 9, the Lagrange multiplier λ in Eq. (1) is crucial; high values ensure uniform distribution, while low values emphasize dataflow. We manually choose and fix λ throughout placement.

Our algorithm primarily requires tuning γ and λ. Our main results in Table I are slightly tuned manually. We further employ Bayesian optimization for automatic tuning and promote the performance, as detailed in Sec. IV-D.

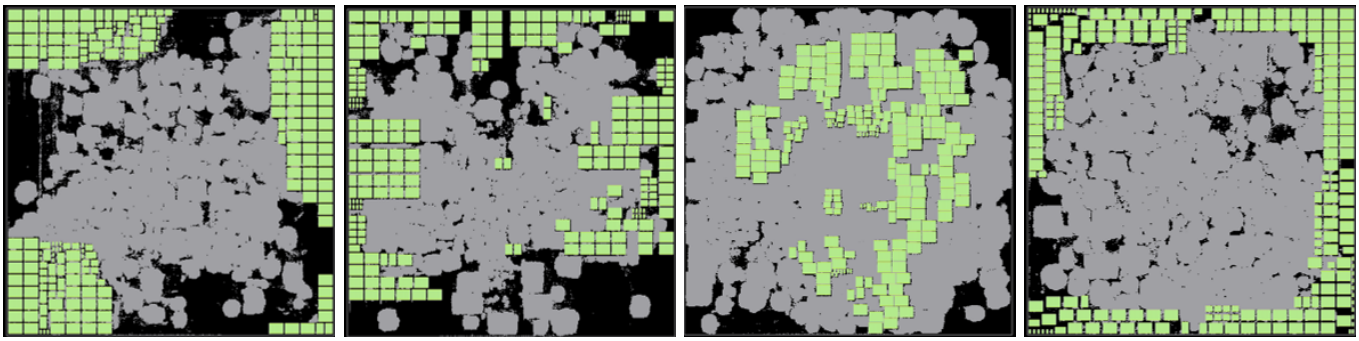
IV. Experimental Results

We implement ReMaP in Python and evaluate it on a Linux server with a 52-core Intel Xeon CPU (2.60 GHz), an NVIDIA RTX 2080S GPU, and 128GB RAM. We use eight RTL designs from the OpenROAD-flow-scripts[2] (ORFS) repository, synthesized with the Nangate45 library. For timing

[2]https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts

TABLE I: Post-route PPA results on eight designs. Best results of WNS and TNS are in **bold**. Freq. and Util. represent the frequency and area utilization ratio of the design, respectively.

Benchmark	Macros (#)	Cells (#)	Nets (#)	Freq. (MHz)	Util.	Methods	rWL (m)	WNS (ns)	TNS (ns)	Power (W)	Overflow (#)	Runtime (s)
ariane133	132	168K	204K	250.0	0.35	*RTL-MP*	6.76	-1.11	-2839.00	0.314	2055	1753
						Hier-RTLMP	7.07	-1.17	-2967.29	0.316	0	572
						DREAMPlace	7.10	-1.33	-3628.90	0.325	23	36
						ReMaP (ours)	7.48	**-1.07**	**-2615.74**	0.318	24	478
ariane136	136	171K	209K	333.3	0.36	*RTL-MP*	8.16	-2.07	-6315.44	0.479	332	4859
						Hier-RTLMP	7.91	-2.23	-6830.66	0.479	100	153
						DREAMPlace	7.88	-2.21	-6700.25	0.483	78	47
						ReMaP (ours)	7.81	**-2.04**	**-6085.84**	0.464	7	589
black_parrot	24	307K	359K	500.0	0.45	*RTL-MP*	9.72	-4.98	-1102.72	0.475	6281	18
						Hier-RTLMP	10.84	-4.75	-2111.50	0.463	75323	124
						DREAMPlace	10.52	-5.15	-1216.80	0.457	6922	45
						ReMaP (ours)	9.34	**-4.67**	**-1090.18**	0.454	4548	322
bp_be	10	51K	66K	1250.0	0.45	*RTL-MP*	3.36	-0.98	-125.01	0.134	25978	2
						Hier-RTLMP	3.35	-0.96	-105.89	0.135	80552	17
						DREAMPlace	3.80	-0.92	-111.73	0.139	23007	24
						ReMaP (ours)	3.89	**-0.90**	**-98.00**	0.136	31288	389
bp_fe	11	33K	43K	555.6	0.47	*RTL-MP*	2.16	-0.41	-19.88	0.156	8841	2
						Hier-RTLMP	2.17	-0.45	-23.68	0.155	34839	17
						DREAMPlace	2.58	-0.65	-43.28	0.162	28885	18
						ReMaP (ours)	2.65	**-0.27**	**-6.88**	0.160	15936	401
bp_multi	26	152K	179K	1250.0	0.47	*RTL-MP*	5.17	-5.84	-6981.22	0.609	237	20
						Hier-RTLMP	5.24	-6.18	-6677.56	0.615	2539	72
						DREAMPlace	5.99	-5.95	-6735.01	0.625	29477	26
						ReMaP (ours)	4.98	**-5.82**	**-6190.78**	0.614	160	491
bp_quad	220	1135K	1448K	384.6	0.39	*RTL-MP*	55.35	-1.40	-22767.70	1.803	161655	10918
						Hier-RTLMP	44.68	-0.99	-16445.80	1.805	11330	1270
						DREAMPlace	51.92	-1.31	-21669.80	1.824	28822	120
						ReMaP (ours)	54.30	**-0.93**	**-14139.20**	1.812	4629	1785
swerv_wrapper	28	98K	117K	500.0	0.61	*RTL-MP*	4.46	-1.36	-1318.75	0.256	15624	17
						Hier-RTLMP	5.25	-1.79	-1705.52	0.261	62726	38
						DREAMPlace	5.33	-1.41	-1364.19	0.262	47836	22
						ReMaP (ours)	4.44	**-1.12**	**-979.03**	0.256	20072	459
Average Rank of *ReMaP* (ours)							2.375	1	1	2.125	1.75	3.5



(a) *RTL-MP* (b) *Hier-RTLMP* (c) *DREAMPlace* (d) *ReMaP* (ours)

Fig. 4: Placement visualization of design bp_quad.

analysis, we increase the frequency of cases without violated timing paths. Then we place macros using different placers, followed by global placement, clock tree synthesis, and routing using OpenROAD [1]. The released version[3] of DREAMPlace 4.1.0 [2] is used as our mixed-size placer for prototyping.

A. Main Results

Table I compares post-route PPA metrics of three baselines and our ReMaP method. We use RTL-MP [9] and Hier-RTLMP [10] with default settings from ORFS, and DREAMPlace 4.1.0 with macro placement enhancements [2]. ReMaP achieves the best WNS and TNS in all eight cases, with average improvements of 8.39% and 16.57%, and maximum improvements of 34.15% and 65.39% for WNS and TNS compared to the strongest baseline on each case. ReMaP also performs above average in routing wirelength, power, and routing overflow, excelling in overflow due to the regularly placed macros.

Fig. 4 visualizes the placement results. RTL-MP shows regular placement but poor timing, while Hier-RTLMP improves timing but lacks regularity with large notch regions.

[3]https://github.com/limbo018/DREAMPlace/releases/tag/4.1.0

TABLE II: Ablation results.

Benchmark	Methods	WNS (ns)	TNS (ns)	Overflow (#)
ariane133	*Fix_Density_0.5*	-1.20	-3051.71	121
	Fix_Density_0.91	**-1.06**	-2636.54	9
	Overlap_Criteria	**-1.06**	-2658.09	18
	Default	-1.07	**-2615.74**	24
ariane136	*Fix_Density_0.5*	-2.14	-6385.50	6
	Fix_Density_0.91	-2.09	-6260.21	2
	Overlap_Criteria	-2.23	-6726.42	4
	Default	**-2.04**	**-6085.84**	7
black_parrot	*Fix_Density_0.5*	-4.82	-1278.52	5669
	Fix_Density_0.91	**-4.66**	-1078.88	2535
	Overlap_Criteria	-5.09	**-880.59**	977
	Default	-4.67	-1090.18	4548
bp_be	*Fix_Density_0.5*	**-0.89**	**-87.29**	30745
	Fix_Density_0.91	-1.02	-118.58	36056
	Overlap_Criteria	-0.91	-100.52	31225
	Default	-0.90	-98.00	31288
bp_fe	*Fix_Density_0.5*	-0.55	-36.08	8843
	Fix_Density_0.91	-0.37	-19.68	2627
	Overlap_Criteria	-0.65	-43.50	7684
	Default	**-0.27**	**-6.88**	15936
bp_multi	*Fix_Density_0.5*	-5.85	-7021.07	493
	Fix_Density_0.91	-6.21	-6376.50	471
	Overlap_Criteria	**-5.67**	**-5937.93**	26
	Default	-5.82	-6190.78	160
bp_quad	*Fix_Density_0.5*	-0.95	-16328.90	4298
	Fix_Density_0.91	-1.40	-17404.60	8417
	Overlap_Criteria	-1.01	-16952.10	5784
	Default	**-0.93**	**-14139.20**	4629
swerv_wrapper	*Fix_Density_0.5*	**-1.12**	-1035.99	31050
	Fix_Density_0.91	-1.37	-1349.50	20305
	Overlap_Criteria	-1.13	-1061.89	21970
	Default	**-1.12**	**-979.03**	20072

DREAMPlace struggles with timing and core congestion. Our ReMaP achieves both superior timing and regularity.

B. Ablation Study

Section III-B discusses our choice of dynamic target density for mixed-size prototyping, which decreases from 0.91 to 0.5 across all eight cases. We also explore fixed densities of 0.5 and 0.91. Additionally, we propose two criteria for macro relocating order in Section III-D. By default, we prioritize weaker dataflow strength for better timing performance, but we also evaluate an overlap criterion for more regular placements. As shown in Table II, while the ablation methods occasionally excel, the default settings generally perform best. Given the variability in macro placement challenges, it's clear that no single fixed setting can address every scenario effectively.

C. Runtime Analysis

In Table I, we compare the runtime of our ReMaP method with baselines. For smaller cases, as shown in Fig. 5(a), a significant portion of our runtime is due to gradient calculations during prototyping. This is an inherent limitation of our approach owing to recursive prototyping. However, for larger cases like bp_quad, as shown in Fig. 5(b), our method's runtime does not escalate steeply, thanks to DREAMPlace's efficient gradient propagation. Despite the case scale increasing by over ten times, the absolute time spent on gradients only increases by about 12.5%. The majority of the runtime, however, is consumed by benchmark I/O, primarily due to DREAMPlace's inefficiency with recursive calls. We are working on improving the I/O efficiency of our framework, and these engineering challenges will be addressed soon. Our method shows potential for greater efficiency in modern, large-scale designs.

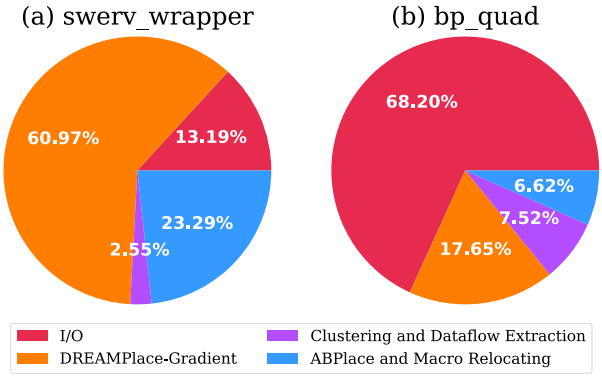


Fig. 5: Runtime breakdown of ReMaP on two representative designs swerv_wrapper and bp_quad.

TABLE III: Results of automatic parameter tuning.

Benchmark	Methods	Params		Metrics	
		γ	λ	WNS	TNS
ariane133	*hand-crafted*	0.943	0.006	-1.07	-2615.74
	50-round BO	0.956	0.001	**-1.02**	**-2484.17**
bp_be	*hand-crafted*	0.943	0.006	-0.90	-98.00
	50-round BO	0.990	0.440	**-0.78**	**-76.26**

D. Parameter Auto-tuning

As highlighted in Sec. III-E, our ReMaP method requires tuning of only two parameters: the decay factor γ and the Lagrange multiplier λ. While we manually adjust these parameters for our main results, we also explore their full potential using Bayesian optimization (BO) for automatic tuning. Table III shows that after 50 rounds of BO, there is an average improvement of 8.75% on WNS and TNS metrics in the ariane133 and bp_be cases compared to our manually tuned parameters.

V. Conclusion

In this work, we introduce and open-source the ReMaP framework, which optimizes macro placement through recursively prototyping and relocating. Our ABPlace method uses angle-based distribution to place macros on an ellipse, followed by periphery-guided relocating, yielding regular layouts with superior timing performance, as confirmed by extensive experiments. While recursive prototyping may reduce efficiency in smaller cases, ReMaP shows promise for scaling to larger cases after addressing engineering challenges.

VI. Acknowledgement

This work was supported by the National Science and Technology Major Project (2022ZD0116600), Jiangsu Science Foundation Leading-edge Technology Program (BK20232003), National Science Foundation of China (62276124, 624B2069), Fundamental Research Funds for the Central Universities (14380020), and Collaborative Innovation Center of Novel Software Technology and Industrialization. Chao Qian is the corresponding author.

REFERENCES

[1] T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem *et al.*, "Toward an open-source digital flow: First learnings from the openroad project," in *Proceedings of ACM/IEEE Design Automation Conference*, Las Vegas, NV, 2019, pp. 1–4.

[2] Y. Chen, Z. Wen, Y. Liang, and Y. Lin, "Stronger mixed-size placement backbone considering second-order information," in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, San Francisco, CA, 2023, pp. 1–9.

[3] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "RePlAce: Advancing solution quality and routability validation in global placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 9, pp. 1717–1730, 2018.

[4] R. Cheng and J. Yan, "On joint learning for solving placement and routing in chip design," in *Advances in Neural Information Processing Systems*, Virtual, 2021.

[5] M. Fogaca, A. B. Kahng, E. Monteiro, R. Reis, L. Wang, and M. Woo, "On the superiority of modularity-based clustering for determining placement-relevant clusters," *Integration*, vol. 74, pp. 32–44, 2020.

[6] Z. Geng, J. Wang, Z. Liu, S. Xu, Z. Tang, M. Yuan, J. Hao, Y. Zhang, and F. Wu, "Reinforcement learning within tree search for fast macro placement," in *Proceedings of the International Conference on Machine Learning*, Vienna, Austria, 2024.

[7] X. Hong, G. Huang, Y. Cai, J. Gu, S. Dong, C.-K. Cheng, and J. Gu, "Corner block list: An effective and efficient topological representation of non-slicing floorplan," in *Proceedings of IEEE/ACM International Conference on Computer Aided Design*, San Jose, CA, 2000, pp. 8–12.

[8] A. B. Kahng, S. Kang, S. Kundu, K. Min, S. Park, and B. Pramanik, "PPA-relevant clustering-driven placement for large-scale VLSI designs," in *Proceedings of ACM/IEEE Design Automation Conference*, San Francisco, CA, 2024.

[9] A. B. Kahng, R. Varadarajan, and Z. Wang, "RTL-MP: toward practical, human-quality chip planning and macro placement," in *Proceedings of the International Symposium on Physical Design*, New York, NY, 2022, pp. 3–11.

[10] A. B. Kahng, R. Varadarajan, and Z. Wang, "Hier-RTLMP: A hierarchical automatic macro placer for large-scale complex ip blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1552–1565, 2023.

[11] A. B. Kahng and Z. Wang, "DG-RePlAce: A dataflow-driven gpu-accelerated analytical global placement framework for machine learning accelerators," *arXiv:2404.13049*, 2024.

[12] Y. Lai, J. Liu, Z. Tang, B. Wang, J. Hao, and P. Luo, "ChiPFormer: Transferable chip placement via offline decision transformer," in *Proceedings of the International Conference on Machine Learning*, Honolulu, HI, 2023.

[13] Y. Lai, Y. Mu, and P. Luo, "MaskPlace: Fast chip placement via reinforced visual representation learning," in *Advances in Neural Information Processing Systems*, Virtual, 2022.

[14] J.-M. Lin, Y.-L. Deng, S.-T. Li, B.-H. Yu, L.-Y. Chang, and T.-W. Peng, "Regularity-aware routability-driven macro placement methodology for mixed-size circuits with obstacles," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 1, pp. 57–68, 2019.

[15] J.-M. Lin, Y.-L. Deng, Y.-C. Yang, J.-J. Chen, and P.-C. Lu, "Dataflow-aware macro placement based on simulated evolution algorithm for mixed-size designs," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 5, pp. 973–984, 2021.

[16] J.-M. Lin, W.-F. Huang, Y.-C. Chen, Y.-T. Wang, and P.-W. Wang, "DAPA: A dataflow-aware analytical placement algorithm for modern mixed-size circuit designs," in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, Munich, Germany, 2021, pp. 1–8.

[17] J.-M. Lin, S.-T. Li, and Y.-T. Wang, "Routability-driven mixed-size placement prototyping approach considering design hierarchy and indirect connectivity between macros," in *Proceedings of ACM/IEEE Design Automation Conference*, Las Vegas, NV, 2019, pp. 1–6.

[18] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled gpu acceleration for modern VLSI placement," in *Proceedings of ACM/IEEE Design Automation Conference*, Las Vegas, NV, 2019, pp. 1–6.

[19] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[20] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "VLSI module placement based on rectangle-packing by the sequence-pair," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 12, pp. 1518–1524, 1996.

[21] J. K. Ousterhout, "Corner stitching: A data-structuring technique for VLSI layout tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 3, no. 1, pp. 87–100, 1984.

[22] Y. Pu, T. Chen, Z. He, C. Bai, H. Zheng, Y. Lin, and B. Yu, "IncreMacro: Incremental macro placement refinement," in *Proceedings of the International Symposium on Physical Design*, New York, NY, 2024, pp. 169–176.

[23] Y. Shi, K. Xue, S. Lei, and C. Qian, "Macro placement by wire-mask-guided black-box optimization," in *Advances in Neural Information Processing Systems*, New Orleans, LA, 2023.

[24] A. Vidal-Obiols, J. Cortadella, J. Petit, M. Galceran-Oms, and F. Martorell, "RTL-aware dataflow-driven macro placement," in *Proceedings of Design, Automation & Test in Europe Conference & Exhibition*, Florence, Italy, 2019, pp. 186–191.

[25] M.-C. Wu and Y.-W. Chang, "Placement with alignment and performance constraints using the B*-tree representation," in *Proceedings of IEEE International Conference on Computer Design*, San Jose, CA, 2004, pp. 568–571.

[26] K. Xue, R.-T. Chen, X. Lin, Y. Shi, S. Kai, S. Xu, and C. Qian, "Reinforcement learning policy as macro regulator rather than macro placer," in *Advances in Neural Information Processing Systems*, Vancouver, Canada, 2024.

[27] J. Z. Yan, N. Viswanathan, and C. Chu, "Handling complexities in modern large-scale mixed-size placement," in *Proceedings of ACM/IEEE Design Automation Conference*, San Francisco, CA, 2009, pp. 436–441.

[28] X. Zhao, T. Wang, R. Jiao, and X. Guo, "Standard cells do matter: Uncovering hidden connections for high-quality macro placement," in *Proceedings of Design, Automation & Test in Europe Conference & Exhibition*, Valencia, Spain, 2024, pp. 1–6.